CSCI 650 - Algorithms and Computability
Exam 2 - Written and Coding Questions

Solutions to the written portion of the exam along with relevant figures should be submitted via PDF to Canvas. Make sure to **justify your answers**. Code should be submitted on INGInious. The exam will close on **October 18th at 11:59 pm**.

You must complete solutions to these problems **individually**. You may ask and answer questions on Discord or Canvas meant to clarify the problems but you may not discuss or post solutions. You are allowed to use your notes, material posted on Canvas, and the textbook (*Introduction to Algorithms*, 3rd edition) during the exam. You may also reference documentation related to specific syntax or function parameters and output (not source code) if necessary. However, you **are not permitted** to use any other resources. This includes ChatGPT, Google, Stack Overflow, and all other tools, websites, and documents available online.

Good luck!

Problems

1. The dominating set problem is defined as follows.

   Definition (Dominating Set)**.** *Given a graph $G = (V, E)$, a* ***dominating set*** *is a set $D \subseteq V$ such that, for all $v \in V$, either $v \in D$ or $\{u, v\} \in E$ where $u \in D$.*

   The Dominating Set Problem
   Input: A graph $G = (V, E)$ and an integer k.
   Output: True if and only if there is a dominating set D of G such that $|D| \leq k$.

   (a) (10 pts) Write a function `checkDominatingSet(G, D, k)` that returns `True` if D is a dominating set of G of size k or less and `False` otherwise. Note that G may not be connected. Submit your code on INGInious and include a code snippet of this function in your PDF submission. `dominatingSetTests.py`, a script meant to help with local testing, is available on Canvas. What is the runtime of this function?

   (b) (5 pts) Complete an argument that dominating set is NP-complete by selecting a known NP-complete problem (one that we have discussed in class) and writing pseudocode for a polynomial time function solving this problem assuming access to a function `findDominatingSet(G)` that is guaranteed to return a minimum dominating set of G. What is the runtime of this function?

2. Implement a greedy approximation for the dominating set problem.

   (a) (10 pts) Write a function `greedyDominatingSet(G)` that uses a greedy approach to find a dominating set of `G`. Submit your code on INGInious and include a code snippet of this function in your PDF submission. `dominatingSetTests.py`, a script meant to help with local testing, is available on Canvas. What is the runtime of this function?

   (b) (5 pts) Come up with a small specific example showing that the greedy algorithm does not guarantee a minimal dominating set. Show a dominating set of minimum size and show the sequence of steps that the greedy algorithm would take.

3. Use genetic algorithms to find small dominating sets. Answer each of the following questions **briefly** but completely.

   (a) (2 pts) What do the individuals in the population look like? What does each piece mean? Include a code snippet of a function to create a random individual.

   (b) (2 pts) How are mutation and recombination achieved? Include code snippets of a selection function and a recombination function.

   (c) (3 pts) How is fitness evaluated? Include a code snippet of a function for calculating the fitness of a single individual.

   (d) (3 pts) Create a figure showing the maximum, average, and minimum fitness values for a population of size 50 over 1000 generations trying to find a dominating set on a path of length 100. Depending on your approach, fitness values may not increase smoothly. Describe and interpret your observations.

Solutions

1(a). Verifying a dominating set

Let $n = |V|$ and $m = |E|$. First check the size and that every chosen vertex belongs to the graph. Then take the union of the chosen vertices' closed neighborhoods. The answer is true exactly when that union is all of V. Isolated vertices are included in H and must themselves be selected; connectivity is not assumed.

```
def checkDominatingSet(G, D, k):
  #you may, but are not required to, convert G to a networkx graph
  H = nx.Graph()
  H.add_edges_from(G[1])
  H.add_nodes_from(G[0])

  chosen = set(D)
  if len(chosen) > k or not chosen <= set(H.nodes()):
    return False
  covered = set(chosen)
  for u in chosen:
    covered.update(H.neighbors(u))
  return len(covered) == len(H)
```

If the function returns true, each vertex is selected or adjacent to a selected vertex, and the size bound holds. Conversely, a dominating set of size at most k passes the size and membership checks and covers every vertex, so the function returns true. With hash sets and adjacency lists, the expected runtime is $O(n + m + |D|)$, or $O(n + m)$ when $D \subseteq V$; auxiliary space is $O(n + m)$ including the NetworkX graph. The neighbor scans total at most $2m$.

1(b). NP-completeness via Vertex Cover

Dominating Set is in NP: a proposed vertex set is a certificate checked by part (a) in polynomial time. To prove NP-hardness, reduce the decision version of Vertex Cover. The construction below applies to simple undirected graphs. First remove isolated vertices, which never need to be in a vertex cover. Retain every original edge uv, and add a fresh vertex w_{uv} joined to both u and v. Thus each original edge becomes a triangle, while retaining its original edge.

```
VertexCoverUsingDominatingSet(G, k):
    if k < 0: return False
    U = vertices of G with positive degree
    H = graph with vertices U and all original edges
    for each edge {u,v} in G:
        create a fresh vertex w_uv in H
        add edges {u,w_uv} and {v,w_uv} to H
    D = findDominatingSet(H)
    return (|D| <= k)
```

If C is a vertex cover of G, it dominates every w_{uv} because it contains u or v. It also dominates every original nonisolated vertex: a vertex outside C has a neighbor in C along any incident edge. Thus C dominates H.

Conversely, take any dominating set D of H. Replace each chosen w_{uv} with u, discarding duplicates. This does not increase its size or lose domination: $N_H[w_{uv}] = \{w_{uv}, u, v\} \subseteq N_H[u]$, since uv was retained. The resulting set C consists only of original vertices. Each w_{uv} must now have u or v in C, so C covers every original edge. Hence G has a cover of size at most k if and only if H has a dominating set of size at most k. An edgeless input produces an empty H, correctly accepting exactly when $k \geq 0$.

The constructed graph has at most $n+m$ vertices and exactly $3m$ edges. Construction and comparison take $O(n + m)$ time and space, with one oracle call. If its time is $T_{\mathrm{DS}}(N, M)$, total time is $O(n + m) + T_{\mathrm{DS}}(n' + m, 3m)$, where $n' \leq n$ counts nonisolated vertices. Thus a polynomial-time minimum-dominating-set algorithm would yield a polynomial-time Vertex Cover algorithm. Together with NP membership, this establishes NP-completeness of the decision problem.[1]

[1] Vertex Cover is the selected known NP-complete problem; the supplied material does not identify which problems were covered in class.

2(a). Greedy dominating set

At each step choose a vertex whose closed neighborhood contains the largest number of currently undominated vertices. Mark those vertices dominated and repeat. Ties use NetworkX insertion order. In the preserved setup edges are inserted before the explicit vertex list, so edge order can determine this order.

```
def greedyDominatingSet(G):
  #you may, but are not required to, convert G to a networkx graph
  H = nx.Graph()
  H.add_edges_from(G[1])
  H.add_nodes_from(G[0])

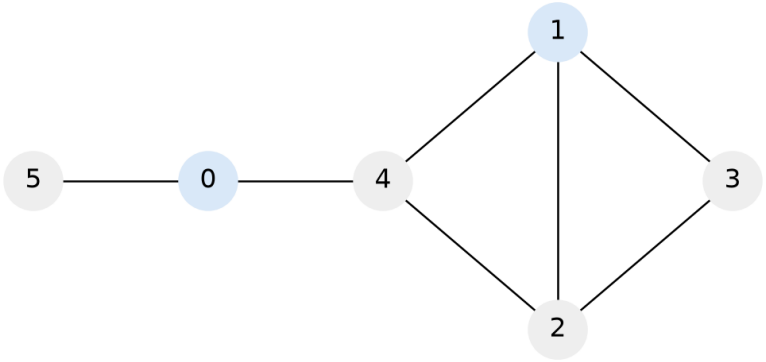
  uncovered = set(H.nodes())
  closed = {v: set(H.neighbors(v)) | {v} for v in H}
  result = []
  while uncovered:
    # max keeps the first vertex in insertion order on a tie.
    v = max(H.nodes(), key=lambda u: len(closed[u] & uncovered))
    result.append(v)
    uncovered.difference_update(closed[v])
  return result
```

While any vertex remains uncovered, some candidate covers at least itself, so each iteration makes progress. The process stops in at most n iterations and returns a dominating set. Each iteration evaluates all closed neighborhoods; their total size is $n + 2m$, so the expected runtime is $O(n(n + m))$ with hash sets. Space is $O(n + m)$.

2(b). A counterexample, including the tie rule

Let $V = [0, 1, 2, 3, 4, 5]$ and give the edges in this order:

$$E = [(0, 4), (0, 5), (1, 2), (1, 3), (1, 4), (2, 3), (2, 4)].$$



The setup inserts vertices in order $[0, 4, 5, 1, 2, 3]$. Initially vertices $4, 1, 2$ each cover four vertices, so the tie rule selects 4. Its closed neighborhood is $\{0, 1, 2, 4\}$, leaving $\{3, 5\}$ uncovered. Every candidate that covers an uncovered vertex now gains one; vertex 0 is first in insertion order, so it is selected and covers 5. Finally 1 is the first candidate covering the remaining vertex 3. The returned sequence is $[4, 0, 1]$.

But $\{0, 1\}$ dominates the graph: $N[0] = \{0, 4, 5\}$ and $N[1] = \{1, 2, 3, 4\}$. No vertex has degree five, so no single vertex dominates all six vertices; the minimum size is two. Moreover, 4 can be removed from the greedy output. Thus this example fails both to be minimum (smallest cardinality) and to be minimal (no proper dominating subset). The distinction matters because the question uses the word “minimal.”

3(a). Individuals

An individual is a binary list of length n. Bit i is one exactly when the ith vertex in the fixed NetworkX node order is selected. Each random bit is independently zero or one with equal probability. Arbitrary vertex labels are allowed because list positions, rather than labels, are the genes.

```
def randomIndividual(G):
  # Position i represents the ith vertex in G.nodes() order.
  n = len(G.nodes()) if isinstance(G, nx.Graph) else len(G[0])
  return np.random.randint(0, 2, size=n).tolist()
```

3(b). Selection, recombination, and mutation

Roulette-wheel selection samples two parents independently with replacement, each with probability proportional to its positive fitness. Single-point crossover swaps suffixes after a uniformly chosen interior cut, producing two children. Empty and one-gene individuals are copied. Mutation flips each child bit independently with probability $1/n$ (using $1/\max(1, n)$ for the empty graph). Parent copies prevent accidental modification of the current population. One unchanged elite is retained each generation.

```
def selection(pop, fitnesses):
  weights = np.asarray(fitnesses, dtype=float)
  if len(pop) == 0 or len(weights) != len(pop):
    raise ValueError("Population and fitnesses must be nonempty and aligned")
  if not np.all(np.isfinite(weights)) or np.any(weights < 0):
    raise ValueError("Fitness weights must be finite and nonnegative")
  total = weights.sum()
  probabilities = weights / total if total > 0 else None
  i, j = np.random.choice(len(pop), size=2, replace=True, p=probabilities)
  return list(pop[i]), list(pop[j])
```

```
def recombine(a, b):
  if len(a) != len(b):
    raise ValueError("Parents must have equal lengths")
  if len(a) < 2:
    return list(a), list(b)
  cut = np.random.randint(1, len(a))
  return list(a[:cut]) + list(b[cut:]), list(b[:cut]) + list(a[cut:])
```

```
def mutate(ind, p=0.1):
  if not 0 <= p <= 1:
    raise ValueError("Mutation probability must be between 0 and 1")
  for i in range(len(ind)):
    if np.random.random() < p:
      ind[i] = 1 - ind[i]
  return ind
```

3(c). Fitness

Let s be the number of selected vertices and u the number of undominated vertices. Maximize

$$f = \frac{1}{1 + s + (n+1)u}.$$

This is strictly positive, so it can be used directly in roulette selection. Every feasible individual has fitness at least $1/(n+1)$, whereas any infeasible individual has fitness at most $1/(n+2)$. Thus every dominating set outranks every infeasible individual. Among dominating sets, fewer selected vertices gives strictly higher fitness. Evaluation takes $O(n+m)$ expected time.

```
def evalFitness(ind, G):
  H = G if isinstance(G, nx.Graph) else nx.Graph()
  if not isinstance(G, nx.Graph):
    H.add_nodes_from(G[0])
    H.add_edges_from(G[1])
  nodes = list(H.nodes())
  if len(ind) != len(nodes) or any(bit not in (0, 1) for bit in ind):
    raise ValueError("Individual must have one binary gene per vertex")
  chosen = {v for v, bit in zip(nodes, ind) if bit}
  covered = set(chosen)
  for v in chosen:
    covered.update(H.neighbors(v))
  missing = len(nodes) - len(covered)
  return 1.0 / (1 + len(chosen) + (len(nodes) + 1) * missing)
```

Generation update

The following complete driver retains the supplied function interface. It records population statistics before reproduction, so the plotted rows describe populations 0 through 999; the returned solution is evaluated after the final reproduction.

```
def runGA(G, popSize=10, generations=100):
  #you may, but are not required to, convert G to a networkx graph
  H = nx.Graph()
  H.add_edges_from(G[1])
  H.add_nodes_from(G[0])

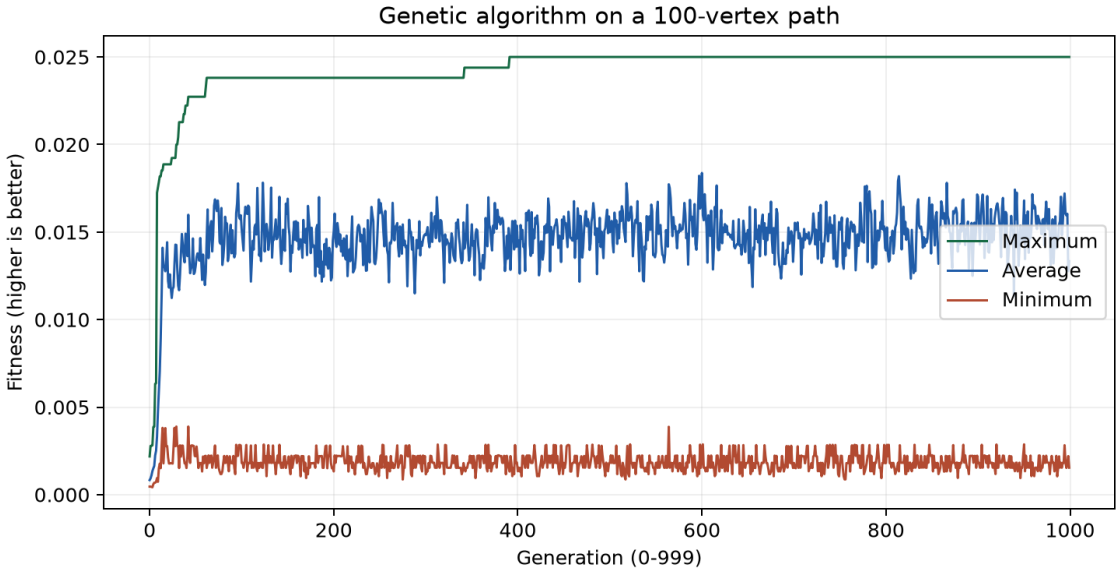
  if popSize < 1 or generations < 0:
    raise ValueError("Population must be positive and generations nonnegative")
  pop = [randomIndividual(H) for _ in range(popSize)]
  fitnessList = [] #each element is (min, avg, max) fitness for a generation

  for gen in range(generations):
    print('Generation', gen)
    fitnesses = [evalFitness(ind, H) for ind in pop]
    fitnessList.append((min(fitnesses), np.mean(fitnesses), max(fitnesses)))
    newPop = []
    # Preserve the best individual and create the remaining population.
    elite = max(range(len(pop)), key=lambda i: fitnesses[i])
    newPop.append(list(pop[elite]))
    while len(newPop) < popSize:
      a, b = selection(pop, fitnesses)
      for child in recombine(a, b):
        mutate(child, p=1.0 / max(1, len(H)))
        newPop.append(child)
        if len(newPop) == popSize:
          break
    pop = newPop

  bestSol, bestFitness = max([(ind, evalFitness(ind, H)) for ind in pop], key=lambda x: x[1])
  return bestSol, bestFitness, fitnessList
```

3(d). Measured experiment

The experiment uses the skeleton's `nx.path_graph(100)`: 100 vertices and 99 edges. Strictly, a path of length 100 has 101 vertices; here the supplied executable setup fixes the intended interpretation. Population size is 50, generations are 1000, and NumPy's random seed is 650. There is no repair step or seeded greedy solution.



The recorded summary statistics are approximately

Generation	Minimum	Average	Maximum
0	0.000460	0.000834	0.002183
999	0.001541	0.013331	0.025000

The returned solution is a valid dominating set of size 39 with fitness $1/40 = 0.025$. Its selected vertices are

> 1, 2, 5, 8, 11, 14, 17, 20, 23, 25, 28, 31, 34, 36, 38, 41, 42, 45, 48, 51, 53, 56, 59, 62, 64, 66, 68, 70, 72, 75, 76, 79, 82, 84, 87, 89, 92, 95, 98.

The maximum rises in steps and never decreases because of elitism. The average improves overall but fluctuates; the minimum remains low and fluctuates because mutation and crossover can create individuals leaving one or more vertices uncovered, which incurs a large penalty. Once feasible solutions appear, their advantage in selection helps the population concentrate on them, while mutation maintains variation. Later plateaus indicate that improvements become less frequent.

This run does not reach the optimum. Each chosen vertex on a path covers at most three vertices, so at least $\lceil 100/3 \rceil = 34$ vertices are necessary. The set $\{1, 4, 7, \ldots, 97, 99\}$ has 34 vertices and dominates the path, proving that bound is achievable. The observed size 39 is five above optimum. A single stochastic run demonstrates behavior but provides no optimality guarantee.

Reproducibility and verification

`run_experiment.py` reproduces the figure and exports all 1000 statistic rows to `fitness_history.csv`, along with the final individual in `experiment_results.json`. `verify.py` compares the verifier with an independent definition on every graph in the NetworkX graph atlas with at most six vertices (33,873 comparisons), checks greedy gains and coverage, and checks GA operators and elitism on empty, disconnected, and path graphs. The supplied testing script is included unchanged.